Assignment – 9 & 10

Code

```python
"""Assignments 9 and 10: variable elimination and belief propagation.

The implementation uses only NumPy for the Student network and the grid MRF.
The optional Alarm demonstration uses pgmpy when it is installed.
"""

from __future__ import annotations

from itertools import product
from math import exp, prod
from time import perf_counter

import numpy as np


CARDINALITY = {
    "Difficulty": 2,
    "Intelligence": 2,
    "Grade": 3,
    "SAT": 2,
    "Letter": 2,
}


class Factor:
    """A dense factor whose axes follow ``variables``."""

    def __init__(self, variables, values, cardinality=None):
        self.variables = tuple(variables)
        self.cardinality = dict(cardinality or CARDINALITY)
        self.values = np.asarray(values, dtype=float)
        expected = tuple(self.cardinality[v] for v in self.variables)
        if self.values.shape != expected:
            raise ValueError(
                f"factor {self.variables} has shape {self.values.shape}; "
                f"expected {expected}"
            )

    @property
    def size(self):
        return int(self.values.size)

    def copy(self):
        return Factor(self.variables, self.values.copy(), self.cardinality)

    def reduce(self, evidence):
        """Fix variables in evidence and return a lower-dimensional factor."""
        values = self.values
```

```
        remaining = list(self.variables)
        for variable, state in evidence.items():
            if variable in remaining:
                axis = remaining.index(variable)
                values = np.take(values, state, axis=axis)
                remaining.pop(axis)
        return Factor(remaining, values, self.cardinality)

    def marginalize(self, variable, operation="sum"):
        if variable not in self.variables:
            return self.copy()
        axis = self.variables.index(variable)
        values = (
            np.sum(self.values, axis=axis)
            if operation == "sum"
            else np.max(self.values, axis=axis)
        )
        remaining = list(self.variables)
        remaining.pop(axis)
        return Factor(remaining, values, self.cardinality)

    def normalize(self):
        total = float(self.values.sum())
        if total <= 0:
            raise ValueError("cannot normalize a factor with zero total")
        return Factor(self.variables, self.values / total, self.cardinality)

    def multiply(self, other):
        variables = tuple(dict.fromkeys(self.variables + other.variables))
        shape = tuple(self.cardinality[v] for v in variables)

        def aligned(factor):
            reshaped = factor.values
            current = list(factor.variables)
            for axis, variable in enumerate(variables):
                if variable in current:
                    old_axis = current.index(variable)
                    if old_axis != axis:
                        reshaped = np.moveaxis(reshaped, old_axis, axis)
                        current.insert(axis, current.pop(old_axis))
                else:
                    reshaped = np.expand_dims(reshaped, axis=axis)
                    current.insert(axis, variable)
            return np.broadcast_to(reshaped, shape)

        return Factor(variables, aligned(self) * aligned(other), self.cardinality)

    def __mul__(self, other):
        return self.multiply(other)


def product_factors(factors):
    if not factors:
        return Factor((), np.array(1.0))
```

```
    result = factors[0]
    for factor in factors[1:]:
        result = result * factor
    return result


def student_factors():
    """Return the standard Student Bayesian-network CPTs as factors."""
    return [
        Factor(("Difficulty",), [0.6, 0.4]),
        Factor(("Intelligence",), [0.7, 0.3]),
        Factor(
            ("Grade", "Difficulty", "Intelligence"),
            [
                [[0.3, 0.05], [0.9, 0.5]],
                [[0.4, 0.25], [0.08, 0.3]],
                [[0.3, 0.7], [0.02, 0.2]],
            ],
        ),
        Factor(("SAT", "Intelligence"), [[0.95, 0.2], [0.05, 0.8]]),
        Factor(("Letter", "Grade"), [[0.1, 0.4, 0.99], [0.9, 0.6, 0.01]]),
    ]


def enumerate_joint(factors, evidence=None):
    """Return the probability of evidence by direct enumeration."""
    evidence = evidence or {}
    variables = tuple(CARDINALITY)
    total = 0.0
    for assignment in product(*(range(CARDINALITY[v]) for v in variables)):
        state = dict(zip(variables, assignment))
        if any(state[v] != value for v, value in evidence.items()):
            continue
        total += prod(
            factor.values[
                tuple(state[v] for v in factor.variables)
            ].item()
            for factor in factors
        )
    return total


def variable_elimination(factors, query, evidence=None, order=None):
    """Compute P(query | evidence), returning the result and largest factor."""
    evidence = evidence or {}
    cardinality = factors[0].cardinality
    working = [factor.reduce(evidence) for factor in factors]
    hidden = [
        variable for variable in cardinality
        if variable not in query and variable not in evidence
    ]
    order = list(order) if order is not None else hidden
    largest = max((factor.size for factor in working), default=1)
    for variable in order:
```

```
        involved = [factor for factor in working if variable in factor.variables]
        if not involved:
            continue
        working = [factor for factor in working if factor not in involved]
        combined = product_factors(involved)
        largest = max(largest, combined.size)
        combined = combined.marginalize(variable)
        largest = max(largest, combined.size)
        working.append(combined)
    result = product_factors(working)
    for variable in tuple(result.variables):
        if variable not in query:
            result = result.marginalize(variable)
    return result.normalize(), largest


def moral_graph(factors):
    graph = {variable: set() for variable in factors[0].cardinality}
    for factor in factors:
        variables = list(factor.variables)
        for variable in variables:
            graph.setdefault(variable, set())
        for i, first in enumerate(variables):
            for second in variables[i + 1:]:
                graph[first].add(second)
                graph[second].add(first)
    return graph


def elimination_order(graph, heuristic):
    graph = {node: set(neighbors) for node, neighbors in graph.items()}
    order = []
    while graph:
        if heuristic == "random":
            node = sorted(graph)[0]
        elif heuristic == "min-degree":
            node = min(graph, key=lambda n: (len(graph[n]), n))
        elif heuristic == "min-fill":
            def fill_count(candidate):
                neighbors = list(graph[candidate])
                return sum(
                    second not in graph[first]
                    for i, first in enumerate(neighbors)
                    for second in neighbors[i + 1:]
                )
            node = min(graph, key=lambda n: (fill_count(n), len(graph[n]), n))
        else:
            raise ValueError(f"unknown ordering heuristic: {heuristic}")
        neighbors = list(graph[node])
        for first in neighbors:
            graph[first].update(set(neighbors) - {first})
            graph[first].discard(node)
        del graph[node]
        order.append(node)
```

```
    return order


def factor_graph_messages(factors, evidence=None, operation="sum", iterations=None):
    """Run sum-product or max-product on a factor graph.

    The Student factor graph is a tree, so one forward/backward pass is enough.
    ``iterations`` is useful for the same routine on a loopy graph.
    """
    evidence = evidence or {}
    cardinality = factors[0].cardinality
    variables = sorted({v for f in factors for v in f.variables})
    neighbors = {v: [] for v in variables}
    for index, factor in enumerate(factors):
        for variable in factor.variables:
            neighbors[variable].append(index)
    v_to_f = {}
    f_to_v = {}
    for variable in variables:
        base = np.eye(cardinality[variable])[evidence[variable]] \
            if variable in evidence else np.ones(cardinality[variable])
        for index in neighbors[variable]:
            v_to_f[(variable, index)] = base / base.sum()
    rounds = iterations or 2 * len(variables)
    combine = np.sum if operation == "sum" else np.max
    for _ in range(rounds):
        new_f_to_v = {}
        for index, factor in enumerate(factors):
            for target in factor.variables:
                values = factor.values.copy()
                for variable in factor.variables:
                    if variable == target:
                        continue
                    message = v_to_f[(variable, index)]
                    shape = [1] * len(factor.variables)
                    shape[factor.variables.index(variable)] = len(message)
                    values = values * message.reshape(shape)
                axis = tuple(
                    i for i, variable in enumerate(factor.variables)
                    if variable != target
                )
                message = combine(values, axis=axis) if axis else values
                message = np.asarray(message, dtype=float)
                new_f_to_v[(index, target)] = message / message.sum()
        new_v_to_f = {}
        for variable in variables:
            for index in neighbors[variable]:
                message = np.ones(cardinality[variable])
                if variable in evidence:
                    message *= np.eye(cardinality[variable])[evidence[variable]]
                for other in neighbors[variable]:
                    if other != index:
                        message *= new_f_to_v[(other, variable)]
                new_v_to_f[(variable, index)] = message / message.sum()
```

```
        f_to_v, v_to_f = new_f_to_v, new_v_to_f
    marginals = {}
    for variable in variables:
        message = np.ones(cardinality[variable])
        if variable in evidence:
            message *= np.eye(cardinality[variable])[evidence[variable]]
        for index in neighbors[variable]:
            message *= f_to_v[(index, variable)]
        marginals[variable] = message / message.sum()
    return marginals


def map_by_enumeration(factors, evidence):
    variables = tuple(CARDINALITY)
    best_assignment, best_score = None, -1.0
    for values in product(*(range(CARDINALITY[v]) for v in variables)):
        assignment = dict(zip(variables, values))
        if any(assignment[v] != state for v, state in evidence.items()):
            continue
        score = np.prod([
            factor.values[tuple(assignment[v] for v in factor.variables)]
            for factor in factors
        ])
        if score > best_score:
            best_assignment, best_score = assignment, float(score)
    return best_assignment, best_score


def grid_factors(size=4, coupling=0.5):
    variables = [f"X{row}{column}" for row in range(size) for column in range(size)]
    cardinality = {variable: 2 for variable in variables}
    factors = []
    potential = np.array([[exp(coupling), exp(-coupling)],
                          [exp(-coupling), exp(coupling)]])
    for row in range(size):
        for column in range(size):
            if row + 1 < size:
                factors.append(Factor(
                    (f"X{row}{column}", f"X{row + 1}{column}"),
                    potential, cardinality))
            if column + 1 < size:
                factors.append(Factor(
                    (f"X{row}{column}", f"X{row}{column + 1}"),
                    potential, cardinality))
    return variables, factors


def exact_grid_marginals(variables, factors):
    result = {}
    order = variables
    for variable in variables:
        marginal, _ = variable_elimination(
            factors, [variable], order=[v for v in order if v != variable]
        )
```

```
        result[variable] = marginal.values
    return result


def plot_grid_errors(errors, path="grid_bp_error.png"):
    """Save the requested convergence plot when Matplotlib is available."""
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("  Plot skipped (install matplotlib to save grid_bp_error.png).")
        return
    plt.figure()
    plt.plot(range(1, len(errors) + 1), errors, marker="o")
    plt.xlabel("Iteration")
    plt.ylabel("Maximum absolute marginal error")
    plt.title("Loopy BP error on 4 x 4 grid MRF")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(path, dpi=150)
    plt.close()
    print("  Error plot saved to", path)


def alarm_demo():
    """Compare orderings on pgmpy's Alarm model when pgmpy is available."""
    try:
        from pgmpy.example_models import load_model
    except ImportError:
        try:
            from pgmpy.utils import get_example_model
        except ImportError:
            print("\nAlarm network: skipped (install pgmpy to run this section).")
            return
        model = get_example_model("alarm")
    else:
        try:
            model = load_model("bnlearn/alarm")
        except ValueError as error:
            print(f"\nAlarm network: unavailable ({error})")
            return
    factors = []
    cardinality = {node: model.get_cardinality(node) for node in model.nodes()}
    for cpd in model.get_cpds():
        factors.append(Factor(
            tuple(cpd.variables), np.asarray(cpd.values), cardinality))
    query = [next(iter(model.nodes()))]
    graph = moral_graph(factors)
    print("\nAlarm network (query:", query[0], ")")
    for heuristic in ("random", "min-degree", "min-fill"):
        order = elimination_order(graph, heuristic)
        start = perf_counter()
        _, largest = variable_elimination(factors, query, order=order)
        print(f"  {heuristic:10s}: largest factor={largest:6d}, "
              f"time={perf_counter() - start:.4f}s")
```

```
    print("  Full enumeration is infeasible because Alarm has 37 variables.")


def main():
    factors = student_factors()
    print("ASSIGNMENTS 9 AND 10: VARIABLE ELIMINATION AND BELIEF PROPAGATION")
    print("=" * 68)
    joint = product_factors(factors)
    print("\nStudent joint table entries:", joint.size)
    print("Joint probability sum:", f"{joint.values.sum():.12f}")
    letter, largest = variable_elimination(factors, ["Letter"])
    print("P(Letter):", np.round(letter.values, 6), "largest factor:", largest)
    for query, evidence in [
        (["Difficulty"], {"Letter": 1}),
        (["Intelligence"], {"Grade": 2, "SAT": 1}),
    ]:
        result, largest = variable_elimination(factors, query, evidence)
        print(f"P({query[0]} | {evidence}) =", np.round(result.values, 6),
              "largest factor:", largest)
        direct = enumerate_joint(factors, evidence)
        print("  enumeration evidence probability:", f"{direct:.12f}")

    print("\nStudent factor-graph marginals:")
    beliefs = factor_graph_messages(factors)
    for variable in CARDINALITY:
        exact, _ = variable_elimination(factors, [variable])
        print(f"  {variable:12s}", np.round(beliefs[variable], 6),
              "max error:", f"{np.max(np.abs(beliefs[variable] - exact.values)):.2e}")

    weak_map, weak_probability = map_by_enumeration(factors, {"Letter": 0})
    max_beliefs = factor_graph_messages(factors, {"Letter": 0}, "max")
    print("\nMax-product with Letter=weak:", weak_map,
          "joint probability:", f"{weak_probability:.8f}")
    print("  max-product Letter belief:", np.round(max_beliefs["Letter"], 6))

    variables, grid = grid_factors()
    exact = exact_grid_marginals(variables, grid)
    print("\n4x4 grid MRF loopy sum-product:")
    errors = []
    for iteration in range(1, 21):
        beliefs = factor_graph_messages(grid, iterations=iteration)
        error = max(np.max(np.abs(beliefs[v] - exact[v])) for v in variables)
        errors.append(error)
        print(f"  iteration {iteration:2d}: maximum error={error:.8f}")
    plot_grid_errors(errors)
    alarm_demo()


if __name__ == "__main__":
    main()
```

Output

```
(.venv) PS D:\SVNIT\Semester - 7\Probabilistic Graphical Models\Practicals\Assignment - 9 & 10> python .\lab9_10.py
ASSIGNMENTS 9 AND 10: VARIABLE ELIMINATION AND BELIEF PROPAGATION
================================================================

Student joint table entries: 48
Joint probability sum: 1.000000000000
P(Letter): [0.432044 0.567956] largest factor: 12
P(Difficulty | {'Letter': 1}) = [0.443379 0.556621] largest factor: 24
  enumeration evidence probability: 0.567956000000
P(Intelligence | {'Grade': 2, 'SAT': 1}) = [0.051983 0.948017] largest factor: 4
  enumeration evidence probability: 0.126580000000

Student factor-graph marginals:
  Difficulty   [0.6 0.4] max error: 1.11e-16
  Intelligence [0.7 0.3] max error: 0.00e+00
  Grade        [0.447  0.2714 0.2816] max error: 5.55e-17
  SAT          [0.725 0.275] max error: 0.00e+00
  Letter       [0.432044 0.567956] max error: 0.00e+00

Max-product with Letter=weak: {'Difficulty': 0, 'Intelligence': 0, 'Grade': 2, 'SAT': 0, 'Letter': 0} joint probability: 0.11850300
  max-product Letter belief: [1. 0.]

4x4 grid MRF loopy sum-product:
  iteration  1: maximum error=0.00000000
  iteration  2: maximum error=0.00000000
  iteration  3: maximum error=0.00000000
  iteration  4: maximum error=0.00000000
  iteration  5: maximum error=0.00000000
  iteration  6: maximum error=0.00000000
  iteration  7: maximum error=0.00000000
  iteration  8: maximum error=0.00000000
  iteration  9: maximum error=0.00000000
  iteration 10: maximum error=0.00000000
  iteration 11: maximum error=0.00000000
  iteration 12: maximum error=0.00000000
  iteration 13: maximum error=0.00000000
  iteration 14: maximum error=0.00000000
  iteration 15: maximum error=0.00000000
  iteration 16: maximum error=0.00000000
  iteration 17: maximum error=0.00000000
  iteration 18: maximum error=0.00000000
  iteration 19: maximum error=0.00000000
  iteration 20: maximum error=0.00000000
  Error plot saved to grid_bp_error.png

Alarm network (query: HISTORY )
  random    : largest factor= 69984, time=0.0022s
  min-degree: largest factor=   144, time=0.0009s
  min-fill  : largest factor=   144, time=0.0015s
  Full enumeration is infeasible because Alarm has 37 variables.
(.venv) PS D:\SVNIT\Semester - 7\Probabilistic Graphical Models\Practicals\Assignment - 9 & 10>
```

Results

